

Merged wording for P0527R1 and P1155R3

Document number: P1825R0

Date: 2019-07-19

Project: Programming Language C++, Core Working Group

Reply-to: David Stone: david@doublewise.net, david.stone@uber.com

P0527R1 ("Implicitly move from rvalue references in return statements" by David Stone) and P1155R3 ("More implicit moves" by Arthur O'Dwyer) both touch the same section of wording, including making some of the same changes. This wording accounts for all of the changes in both papers.

Change 11.9.5 [class.copy.elision]/3:

An *implicitly movable entity* is a variable of automatic storage duration that is either a non-volatile object or an rvalue reference to a non-volatile object type. In the following copy-initialization contexts, a move operation might be used instead of a copy operation:

- If the expression in a return or `co_return` statement ([stmt.return]) is a (possibly parenthesized) *id-expression* that names ~~an object with automatic storage duration an~~ implicitly movable entity declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression*, or
- if the operand of a *throw-expression* is ~~the name of a non-volatile automatic object (other than a function or catch-clause parameter)~~ a (possibly parenthesized) *id-expression* that names an implicitly movable entity whose scope does not extend beyond the ~~end of the innermost enclosing try-block (if there is one)~~ compound-statement of the innermost try-block or function-try-block (if any) whose *compound-statement* or *ctor-initializer* encloses the *throw-expression*,

overload resolution to select the constructor for the copy or the `return_value` overload to call is first performed as if the ~~object were designated by expression or operand were~~ an rvalue. If the first overload resolution fails or was not performed, ~~or if the type of the first parameter of the selected constructor or the return_value overload is not an rvalue reference to the object's type (possibly cv-qualified)~~, overload resolution is performed again, considering the ~~object expression or operand~~ as an lvalue. [Note: This two-stage overload resolution must be performed regardless of whether copy elision will occur. It determines the constructor or the `return_value` overload to be called if elision is not performed, and the selected constructor or ~~the~~ `return_value` overload must be accessible even if the call is elided. — end note]

[Example:

```
void f()_{
    T x;
    try_{
        T y;
        try_{g(x);}
        catch(...)_{
            if(/*...*/)
                throw x;           // does not move
            throw y;               // moves
        }
        g(y);
    } catch(...)_{
        g(x);
        // g(y); // error
    }
}
```

-- end example]

Change Annex C.5: [diff.cpp17.class] to add an additional entry:

Affected subclauses: [class.copy.elision]

Change: A function returning an implicitly-movable entity may invoke a constructor taking an rvalue reference to a type different from that of the returned expression. Function and catch-clause parameters can be thrown using move constructors.

Rationale: Side-effect of making it easier to write more efficient code that takes advantage of moves.

Effect on original feature: Valid C++17 code may become ill-formed in this International Standard. For example:

```
struct base {  
    base();  
    base(base const &);  
private:  
    base(base &&);  
};  
  
struct derived : base {};  
  
base f(base b) {  
    throw b;           // error: base(base &&) is private  
    derived d;  
    return d;          // error: base(base &&) is private  
}
```